
Compilers Project

JADE: Joint Asynchronous Distributed Environment

Team Members

Name	Roll Number	Email
Anup Chavan	CS23BTECH11005	CS23BTECH11005@iiith.ac.in
Arnav A Maiti	CS23BTECH11007	CS23BTECH11007@iiith.ac.in
Bhumin Hirpara	CS23BTECH11009	CS23BTECH11009@iiith.ac.in
Karan Gupta	CS23BTECH11023	CS23BTECH11023@iiith.ac.in
Pranjal Prajapati	CS23BTECH11048	CS23BTECH11048@iiith.ac.in
R Prabhu Teja	CS23BTECH11050	CS23BTECH11050@iiith.ac.in
Roshan Y Singh	CS23BTECH11052	CS23BTECH11052@iiith.ac.in
Sujal Meshram	CS23BTECH11060	CS23BTECH11060@iiith.ac.in

Abstract

Jade is a Domain-Specific Language (DSL) designed to simplify the development of parallel and distributed applications, with a particular focus on high-performance computing (HPC). It abstracts away the intricate and error-prone low-level details of message passing, such as explicit MPI rank management, communicator creation, message serialization, and manual synchronization. Instead, Jade provides a declarative syntax that enables developers to define logical roles (e.g., master, worker), tasks, and communication patterns (broadcast, gather, send/recv) in a clean, human-readable way. The Jade compiler automates the generation of efficient native C++ executables that internally manage MPI calls, communicator setups, and role-to-rank mappings, producing highly optimized binaries suitable for large-scale distributed systems. By removing boilerplate and embedding static checks for deadlocks and communication mismatches, it improves developer productivity and program correctness. Additionally, Jade supports hybrid programming models where users can write performance-critical kernels in C++ or leverage compiler-driven MLIR optimizations. Jade targets use cases where deterministic performance, low-latency communication, and tight orchestration are required, such as synchronous SGD training, scientific simulations, and distributed numerical solvers. It provides a powerful and safe abstraction layer that increases maintainability while allowing fine-grained control and interoperability with existing HPC codebases.

Contents

1	Introduction	3
2	Domain	4
3	Language Specification	5
3.1	Role Declarations	5
3.2	Task Definitions	5
3.3	Communication Operators	5
3.4	Supported Types	6
3.4.1	Primitive Types	6
3.4.2	Composite Types	6
3.5	Example Program: Distributed Matrix Multiplication	6
4	Unique Selling Proposition (USP)	8
5	Comparison with Existing Tools	9
5.1	Jade	9
5.2	MPI	9
5.3	Apache Spark	9
5.4	Ray	9
5.5	Dask	10
5.6	Charm++	10
6	Non-Trivial Input Program Example: Sudoku Checker	11
7	Implementation Overview	13
7.1	Compilation Flow	13
7.2	Role Mapping	13
8	Conclusion	15

1 Introduction

Developing applications for distributed systems poses significant challenges due to the complexity of handling message passing, synchronization, process coordination, and parallel execution management. Traditional approaches like MPI (Message Passing Interface) offer a powerful set of low-level APIs that give developers fine-grained control over communication and parallelism. However, MPI programming is notoriously verbose and error-prone, requiring explicit management of ranks, communicators, message serialization, and deadlock-prone synchronization logic. As a result, writing correct, efficient, and maintainable MPI code demands significant expertise and effort.

Higher-level frameworks such as Apache Spark, Ray, and Dask have emerged to simplify distributed programming by providing abstractions for large-scale data processing and dynamic task scheduling. Spark enables distributed data analytics using resilient distributed datasets (RDDs) and DataFrame APIs but targets batch-oriented workloads and incurs considerable runtime overhead due to its JVM-based architecture and dynamic scheduler. Ray focuses on Python-first task and actor models suitable for machine learning pipelines, but its dynamic scheduler introduces non-negligible overhead, making it suboptimal for tightly-coupled, latency-sensitive HPC workloads. Dask provides an easy-to-use parallel computation model for data scientists by scaling familiar Pandas-like operations, but it emphasizes task scheduling over low-latency collective communication.

Jade introduces a new paradigm by allowing developers to declaratively specify the **what** of parallel and distributed computation instead of the **how**. Programmers describe logical roles (such as master, worker groups) and high-level communication patterns (broadcast, gather, send, receive), while the compiler takes care of translating these abstractions into efficient, deadlock-free, and well-optimized C++ code that uses MPI under the hood.

Key advantages of Jade include:

- **Elimination of Boilerplate:** Developers no longer need to manually manage ranks, communicators, serialization, or message buffers.
- **Productivity:** A simple and intuitive DSL syntax reduces development time and improves code readability.
- **Security:** The compiler statically validates communication correctness, detects potential deadlocks, and verifies role-to-rank consistency.
- **Performance:** Generates native C++/MPI binaries, eliminating interpreter/runtime overhead typical of frameworks like Spark or Ray, enabling predictable low-latency communication.
- **Interoperability:** Allows seamless integration of C++ compute kernels for performance-critical operations, and supports MLIR optimizations for advanced kernel compilation.
- **Reliability:** Ensures consistent execution across distributed nodes with deterministic task scheduling and communication patterns.
- **Robustness:** Built-in error detection mechanisms catch serialization mismatches and communication failures at compile time.

Jade is designed to be especially well-suited for domains where deterministic performance, tightly-coupled communication, and predictable execution patterns are essential, such as synchronous distributed machine learning (e.g., synchronous SGD), large-scale matrix computations, scientific simulations, and parallel numerical solvers.

By enabling developers to focus on the logical structure of their distributed workloads while automating low-level communication intricacies, Jade bridges the gap between productivity and high performance in distributed computing.

2 Domain

Jade is specifically designed to address use cases where deterministic performance, low-latency communication, and tightly-coupled computation are critical. The language excels in environments that require precise orchestration of parallel tasks with predictable communication patterns, avoiding the unpredictability and overhead of dynamic schedulers or runtime-driven frameworks. Its main target domains include:

- **Synchronous Machine Learning Workloads:** Jade is ideal for implementing synchronous stochastic gradient descent (SGD) and distributed deep learning algorithms, where multiple worker nodes compute gradients in parallel and aggregate them deterministically at each training step. The explicit communication model allows for low-latency allreduce or broadcast operations, providing a highly efficient alternative to frameworks like TensorFlow’s parameter server model or PyTorch Distributed Data Parallel that rely on implicit abstractions.
- **Distributed Numerical Solvers and Matrix Computations:** Applications such as distributed sparse linear solvers, dense matrix multiplication, and parallel eigensolvers require fine-grained control of data partitioning and communication. Jade’s role-based abstraction makes it easy to split matrices into row blocks, distribute computation tasks, and gather results without manually managing MPI ranks and communicators, significantly reducing development complexity.
- **Dataflow-Oriented Scientific Pipelines:** Scientific workflows often consist of multiple computation stages, including preprocessing, simulation, and postprocessing, where each stage depends on the results of the previous one. Jade enables developers to describe these workflows declaratively, mapping each stage to tasks and roles while automating communication between them. This makes it well-suited for complex pipelines such as climate modeling, genomics data analysis, and astrophysical simulations.
- **Low-Latency Orchestrated Simulations:** Real-time distributed simulations — for example, network simulators, physical systems simulations, or large-scale agent-based models — require strict timing and low communication latency between compute nodes. Jade’s explicit role and task management, combined with compile-time communication optimization, enables deterministic scheduling and predictable execution across a cluster of nodes.
- **Tightly-Managed Data-Parallel Algorithms:** Algorithms such as MapReduce variants where strict synchronization is required at each step (e.g., sorting, aggregation, graph traversal) can be expressed cleanly in Jade. The compiler generates efficient communication calls, replacing the need for hand-written scatter/gather/reduce operations. This is especially useful in scenarios where each worker’s result contributes directly to the next phase of computation, such as in distributed graph processing (PageRank, BFS).

Jade is not intended for loosely-coupled or embarrassingly parallel tasks where communication overhead is negligible or for applications where fault tolerance and dynamic scaling are the primary concern. Instead, it shines in use cases demanding precise control over communication, strong determinism, and high performance, bridging the gap between manual MPI programming and heavyweight distributed data frameworks.

3 Language Specification

Jade offers a structured, high-level language design that abstracts the complexity of parallel and distributed computing. It enables developers to define the structure of distributed applications declaratively while automating the generation of efficient communication patterns. The key components of the language are role declarations, task definitions, communication operators, and supported data types.

3.1 Role Declarations

```
role master;  
role worker[4];
```

Roles are logical entities that represent a group of compute nodes in the distributed system. Each role is mapped by the compiler to one or more MPI ranks. This abstraction layer removes the need for manual rank management by developers. Roles provide clear semantic meaning to nodes, such as `master` orchestrating the computation and `worker` performing the actual data processing tasks.

Roles can be parameterized to represent multiple identical nodes (e.g., `worker[4]` creates 4 worker roles with ranks automatically assigned).

3.2 Task Definitions

```
task load_data on master {  
    dataset = read_csv("data.csv");  
    broadcast dataset to worker[0..3];  
}
```

Tasks are fundamental units of computation that are explicitly bound to specific roles. A task defines a sequence of high-level operations executed by the nodes assigned to that role.

Key properties of task definitions:

- Runs on a specified role or subset of roles.
- Describes input/output dataflow declaratively.
- Automatically manages the orchestration of communication and computation.

The example above shows a task where the master reads a dataset from disk and broadcasts it to workers.

3.3 Communication Operators

Jade provides a set of built-in communication primitives that abstract low-level MPI functions:

- `broadcast data to role[N..M]`
Transfers data from the calling role to all specified target roles efficiently using `MPI_Bcast` or equivalent collective operations.
- `send data to role[N]`
Sends data directly to the specified role, implemented using `MPI_Send`. Used for point-to-point communication.
- `recv data from role[N]`
Receives data from a specified role, implemented using `MPI_Recv`. Simplifies matching send/recv pairs.
- `gather from role[N..M]`
Collects data from multiple source roles into the calling role. Internally uses `MPI_Gather` or `MPI_Gatherv`.
- `spawn task_name on role[N..M]`
Dynamically triggers the execution of a defined task on specified roles. Facilitates flexible computation pipelines.

3.4 Supported Types

3.4.1 Primitive Types

The language supports fundamental types commonly required for distributed applications:

- `int` - Integer numbers
- `float` - Floating-point numbers
- `bool` - Boolean values
- `string` - Text data

3.4.2 Composite Types

Composite types enable structured and multidimensional data management:

- `tensor<int>[3][3]`
Fixed-size multi-dimensional arrays (tensors). The compiler statically checks dimensions and ensures proper serialization across MPI communications.
- `list<int>`
Variable-length arrays useful for scenarios where the number of elements is not known at compile time.
- `record { field1: int; field2: float; }`
User-defined structured types (similar to structures in C/C++), providing an easy way to represent complex data types with named fields.

3.5 Example Program: Distributed Matrix Multiplication

This example illustrates a typical Jade program where a large matrix multiplication is performed in a distributed manner. It shows clear separation of concerns between data loading, distributed computation, and result aggregation.

Note: `$` is used to denote a comment

```
role master;
role worker[4];

task load on master {
  A = read_tensor("A.csv", [1000, 1000]);
  B = read_tensor("B.csv", [1000, 1000]);
  split A to worker[0..3] strategy=row; $ split is some user-defined function
  broadcast B to worker[0..3];
}

task compute on worker {
  local_A = recv from master;
  B_full = recv from master;
  local_C = matmul(local_A, B_full);
  send local_C to master;
}

task aggregate on master {
  parts = gather from worker[0..3];
  C = assemble(parts); $ assemble is some user-defined function
  write_tensor("C.csv", C);
}
```

Explanation of workflow:

1. **Load Task:** The master reads input matrices A and B from disk, splits A by rows into chunks, and broadcasts B to all workers.
2. **Compute Task:** Each worker receives its local A partition and the full matrix B, computes a local result partition (matrix multiplication), and sends it back to the master.

3. **Aggregate Task:** The master gathers computed parts from all workers, assembles the full result matrix C , and writes it to disk.

This clearly demonstrates the ease of describing distributed data partitioning, communication, and aggregation using the Jade DSL, with no need for explicit MPI calls or rank management.

4 Unique Selling Proposition (USP)

Jade introduces several key innovations that set it apart from existing distributed programming frameworks and DSLs:

- **Role-Based Abstraction:** Rather than exposing low-level MPI rank management, Jade allows developers to define meaningful logical roles such as `master`, `worker` [4], or `parameter_server` [2]. This abstraction provides clear semantic meaning, improves code readability, and reduces the chance of rank-related errors.
- **Collective Operations as First-Class Language Primitives:** Common communication patterns such as broadcast, gather, scatter, and reduction are built directly into the language syntax as first-class constructs. The compiler intelligently maps these to efficient MPI collective operations (e.g., `MPI_Bcast`, `MPI_Gather`, `MPI_Allreduce`), eliminating the need for developers to manually implement them, while ensuring optimized execution.
- **Productivity & Safety Enhancements:** Boilerplate code such as explicit communicator management, rank checking, and manual message serialization is fully eliminated. Jade includes static compile-time analysis that enforces correct communication patterns, ensures role-type matching, and prevents deadlocks by checking for unmatched sends and receives. This makes programs safer and significantly reduces development time.
- **Interoperability with Native C++ Kernels and MLIR:** For performance-critical computational kernels (e.g., matrix multiplication, scientific solvers), Jade supports seamless integration of user-defined C++ functions or modules. Developers can write compute-intensive parts in idiomatic C++, compile them with MLIR for further optimization, and invoke them from within the DSL. This provides both high productivity and maximum performance without sacrificing flexibility.
- **Deterministic Execution Model:** Jade avoids the non-determinism often found in dynamic scheduling frameworks by relying on a static role and task graph. The explicit task structure and communication plan are compiled into efficient MPI binaries, resulting in predictable low-latency behavior suitable for HPC workloads.
- **Extensible Architecture:** The compiler architecture is designed for future extensions, such as automatic fault-tolerance mechanisms, dynamic load balancing, and integration with heterogeneous hardware accelerators (GPUs, TPUs).

5 Comparison with Existing Tools

5.1 Jade

- **Abstraction Level:** Declarative DSL specifying roles, tasks, and communication patterns.
- **Language Integration:** Native C++ kernels with DSL front-end.
- **Runtime Overhead:** Minimal, as programs compile to efficient native MPI executables.
- **Collective Operations:** Built-in language primitives automatically optimized into efficient MPI calls.
- **Debugging Support:** Compile-time static analysis for role mapping and communication correctness.
- **Interoperability:** Supports C++ kernels and MLIR for high-performance integration.
- **Target Use Cases:** HPC simulations, distributed ML pipelines, scientific dataflows with deterministic execution.

5.2 MPI

- **Abstraction Level:** Low-level imperative API requiring manual rank and communicator management.
- **Language Integration:** Pure C/C++ code.
- **Runtime Overhead:** None (direct MPI calls).
- **Fault Tolerance:** None by design; requires manual management.
- **Collective Operations:** Manual calls to `MPI_Bcast`, `MPI_Gather`, etc.
- **Debugging Support:** Manual rank checks and MPI debugging tools.
- **Interoperability:** Direct integration with C/C++.
- **Target Use Cases:** High-performance MPI applications in HPC.

5.3 Apache Spark

- **Abstraction Level:** High-level data processing framework using RDDs and DataFrames.
- **Language Integration:** JVM ecosystem (Scala, Python).
- **Runtime Overhead:** High (due to JVM and dynamic scheduling).
- **Fault Tolerance:** Built-in (RDD lineage and recomputation).
- **Collective Operations:** Built-in RDD operations abstract communication.
- **Debugging Support:** Hides low-level communication details.
- **Interoperability:** JVM-based ecosystem.
- **Target Use Cases:** Batch processing and big data analytics.

5.4 Ray

- **Abstraction Level:** Dynamic task-based distributed computation framework.
- **Language Integration:** Python runtime.
- **Runtime Overhead:** Medium (due to Python runtime).
- **Fault Tolerance:** Built-in task retry and scheduling.
- **Collective Operations:** Library-based (`ray.collect()`).
- **Debugging Support:** Limited abstractions.
- **Interoperability:** Python ecosystem.
- **Target Use Cases:** Distributed ML workloads, actor-based applications.

5.5 Dask

- **Abstraction Level:** Parallel collections and task scheduling framework for Python.
- **Language Integration:** Python runtime.
- **Runtime Overhead:** Medium (task graph management).
- **Fault Tolerance:** Built-in task retry and dynamic execution graph.
- **Collective Operations:** Library-based (`dask.compute()`).
- **Debugging Support:** Python tools with limited distributed trace support.
- **Interoperability:** Python ecosystem.
- **Target Use Cases:** Data science and parallel analysis tasks.

5.6 Charm++

- **Abstraction Level:** Object-based distributed programming using message-driven execution.
- **Language Integration:** C++.
- **Runtime Overhead:** High (complex runtime system).
- **Fault Tolerance:** Built-in checkpointing and fault recovery.
- **Collective Operations:** Library-based.
- **Debugging Support:** Advanced runtime instrumentation.
- **Interoperability:** C++ ecosystem.
- **Target Use Cases:** Parallel applications designed around object decomposition.

Jade bridges the gap between manual MPI programming and heavyweight distributed frameworks by providing a high-level declarative model focused on determinism, safety, and performance in tightly-coupled distributed applications.

6 Non-Trivial Input Program Example: Sudoku Checker

This example demonstrates how Jade enables developers to express a distributed Sudoku validation algorithm in a clean and declarative manner. It illustrates role declarations, task definitions, data distribution, inter-role communication, and result aggregation, all without requiring manual MPI rank or communicator management.

```
role master;
role row_checker[3];
role col_checker[3];
role block_checker[3];

task load_grid on master {
  // Load the 9x9 Sudoku grid from a CSV file
  grid = read_tensor("sudoku.csv", [9,9]);

  // Broadcast the grid to all checker roles in parallel
  broadcast grid to row_checker[0..2], col_checker[0..2], block_checker[0..2];
}

task check_rows on row_checker {
  // Each row_checker checks a portion of the grid (3 rows per worker)
  result = check_row(grid); // Custom C++ function invoked here
  send result to master; // Send boolean result to master
}

task check_cols on col_checker {
  // Each col_checker checks a portion of the grid (3 columns per worker)
  result = check_column(grid); // Custom C++ function invoked here
  send result to master;
}

task check_blocks on block_checker {
  // Each block_checker validates one of the three 3x3 sub-grids
  result = check_block(grid); // Custom C++ function invoked here
  send result to master;
}

task aggregate_results on master {
  // Gather results from all row, column, and block checkers
  row_results = gather from row_checker[0..2];
  col_results = gather from col_checker[0..2];
  block_results = gather from block_checker[0..2];

  // Final validation logic
  if (all(row_results) && all(col_results) && all(block_results)) {
    print("Valid Sudoku");
  } else {
    print("Invalid Sudoku");
  }
}
```

Detailed Workflow Explanation

1. **Role Declaration:** The program defines five distinct roles:

- **master:** Orchestrates the overall flow.
- **row_checker[3]:** Three workers responsible for validating rows.
- **col_checker[3]:** Three workers responsible for validating columns.
- **block_checker[3]:** Three workers responsible for validating 3x3 blocks.

This abstracts away MPI ranks into meaningful names, simplifying development.

2. **Data Loading and Distribution:** The master loads the Sudoku grid as a 9x9 tensor from disk. Instead of manually specifying ranks and buffers, the `broadcast` operator declaratively sends the grid to all checker roles.
3. **Parallel Computation Tasks:** Each checker role runs a dedicated task:
 - `check_rows`: Processes assigned rows and performs row validation logic.
 - `check_cols`: Processes assigned columns.
 - `check_blocks`: Processes assigned sub-grids.The `check_row`, `check_column`, and `check_block` functions are assumed to be efficient, user-defined C++ functions integrated seamlessly into the DSL.
4. **Result Communication:** Results are sent back from each worker role to the master using the `send` operator, removing the need to manually manage MPI communication calls.
5. **Aggregation and Validation:** The master gathers all results with the `gather` from operator and performs final validation in a straightforward if-statement. The result is deterministically computed, and the program prints either "Valid Sudoku" or "Invalid Sudoku".
6. **Compile-Time Optimizations and Safety:** The Jade compiler checks at compile time that all communication matches (every send has a corresponding receive) and generates optimized MPI binaries, replacing all high-level operators with efficient MPI calls like `MPI_Bcast`, `MPI_Gather`, and `MPI_Send/MPI_Recv`.

This example highlights Jade's core strength: declarative, safe, and efficient expression of tightly-coupled parallel workflows without manual communication management or boilerplate code.

7 Implementation Overview

7.1 Compilation Flow

Jade’s compiler translates high-level DSL programs into efficient parallel executables by following these well-defined phases:

1. **Lexical Analysis:** The source code is first tokenized by the lexer into a stream of tokens representing language elements such as keywords (`role`, `task`, `broadcast`, etc.), identifiers, literals, operators, and delimiters. The lexer removes whitespace and comments, and produces well-defined tokens for the parser. Example tokens:
 - `role` → `ROLE_KEYWORD`
 - `master` → `IDENTIFIER`
 - `;` → `SEMICOLON`
 - String literals → `STRING_LITERAL`
2. **Parsing:** The DSL source code is parsed into an Abstract Syntax Tree (AST). The parser validates syntax and ensures correct use of DSL constructs such as `role`, `task`, and communication operators (`broadcast`, `send`, etc.).
3. **Semantic Analysis:** The compiler performs role resolution and type checking. It ensures that:
 - All communication calls are matched correctly (e.g., every `send` has a corresponding `recv`).
 - Role and task declarations are valid and well-scoped.
 - User-defined types (e.g., tensors, lists, records) are well-typed and dimensionally correct.
4. **Optimization Passes:** The compiler applies optimizations such as:
 - Compile-time deadlock detection: Verifying that circular waits or unmatched communication patterns cannot occur.
5. **Code Generation:** The optimized AST is translated into a fully compilable C++ source file embedding appropriate MPI calls:
 - `broadcast` → `MPI_Bcast`
 - `send` → `MPI_Send`
 - `recv` → `MPI_Recv`
 - `gather from` → `MPI_Gatherv`The generated C++ code also includes initialization of MPI and role-to-rank mappings for clarity and debugging.
6. **Compilation to Binary:** Finally, the generated C++ source file is compiled (e.g., using `g++` with MPI libraries) to produce a single binary executable.

7.2 Role Mapping

Jade automatically assigns MPI ranks to roles for simplicity and safety:

- The `master` role is always assigned **MPI rank 0**.
- Subsequent ranks are assigned to worker roles in the order declared in the DSL source.
- For example, with:

```
role master;  
role row_checker[3];  
role col_checker[3];
```

The mapping will be:

- `master` → rank 0

- row_checker[0] → rank 1
- row_checker[1] → rank 2
- row_checker[2] → rank 3
- col_checker[0] → rank 4
- col_checker[1] → rank 5
- col_checker[2] → rank 6

This ensures predictable and deterministic role-to-rank mapping.

8 Conclusion

Jade provides a powerful and intuitive solution for building parallel and distributed applications in domains that demand high performance, deterministic execution, and fine-grained control. By abstracting away low-level complexities such as MPI rank management, explicit communicator setup, and manual serialization, Jade allows developers to focus solely on the core computational logic.

Its declarative syntax for defining roles, tasks, and communication patterns offers simplicity and clarity, reducing development time and lowering the risk of errors commonly found in traditional MPI programming. Compile-time static analysis ensures correctness by preventing deadlocks and mismatched communication calls before execution, improving safety and reliability.

Moreover, the ability to seamlessly integrate optimized C++ kernels and MLIR-compiled compute-heavy sections empowers developers to achieve peak performance without sacrificing flexibility. The resulting executables are efficient native binaries that leverage MPI for low-latency, high-throughput communication across large clusters.

Jade is particularly well-suited for:

- High-performance scientific simulations.
- Synchronous machine learning workloads such as distributed SGD.
- Deterministic dataflow-oriented processing pipelines.
- Low-latency orchestrated simulations and tightly-managed parallel algorithms.

By combining ease of use, safety, and performance, Jade bridges the gap between productivity-focused high-level frameworks and low-level, error-prone MPI programming, making it the ideal solution for developers aiming to build scalable, deterministic distributed systems.